MATLAB implementation OF 6 bit delta sigma modulator (CIFF architecture)

Order =3,No of bits =6,Form =CIFF

Out of band gain =1.5

Input frequency =23

Nfft=16384.Fs=16384

No of bits for truncation =20

OSR=128

Zeros of ntf at z=1

```
snr_manual_calculated =
  135.7656
```

SNR after difference equation implementation

```
snr_after_filtering =
  128.2224
```

After the FIR filtering of order 256

```
snr_after_filtering_trunc =
  127.0408
```

After truncating output and coefficient upto 20 bits

```
snr_after_decimation =
  128.8338
```

SNR after downsampling by OSR=128

```
snr_after_simulation =
  134.4206
```

SNR of waveform generated by simulateDSM function for same ntf

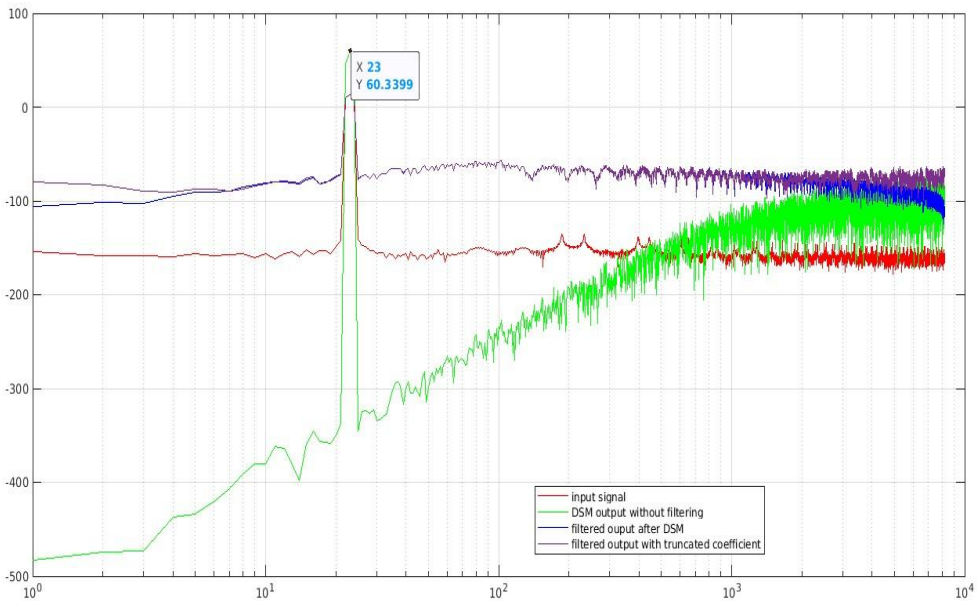

Figure 1: PSD of output signal (without mismatch)

Output spectrum after applying 10 percent mismatch in the unit capacitor

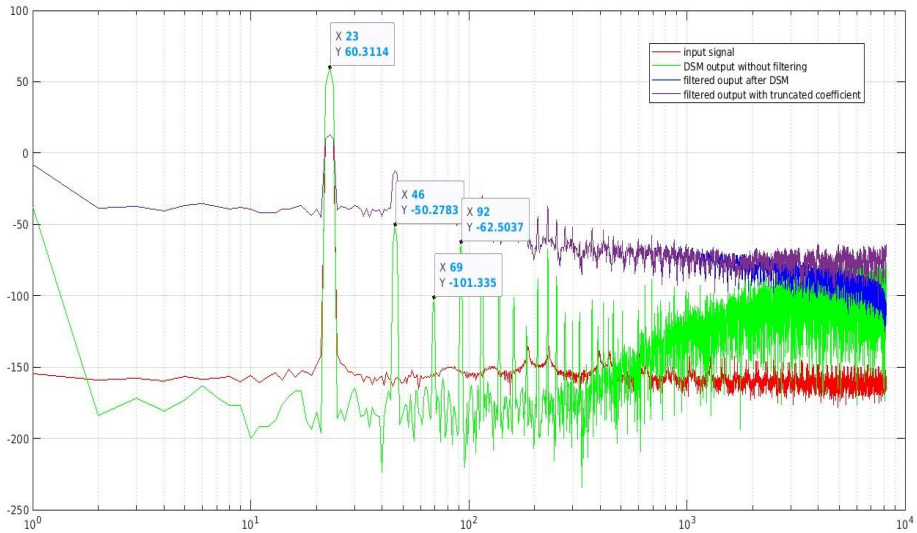

Fig 2:PSD of spectrum with 10 percent mismatch

By applying 10 percent mismatch in unit capacitor we see significant drop in SNDR and addition of spurious tones in output .To visualize the variation of SNR ,2000 no of simulations were carried out.

.

```
snr_manual_calculated =

   48.0278

snr_after_filtering =

   49.9510

snr_after_filtering_trunc =

   49.9511

snr_after_decimation =

   49.9807

snr_after_simulation =

  134.4206
```

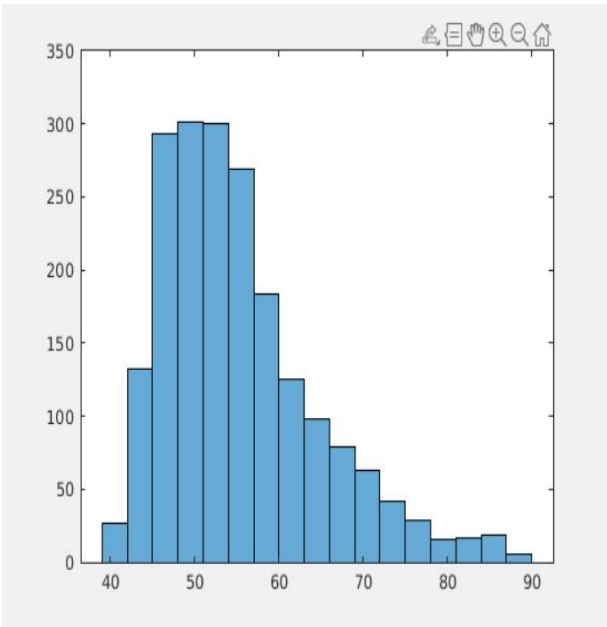

Fig 3:SNR distribution

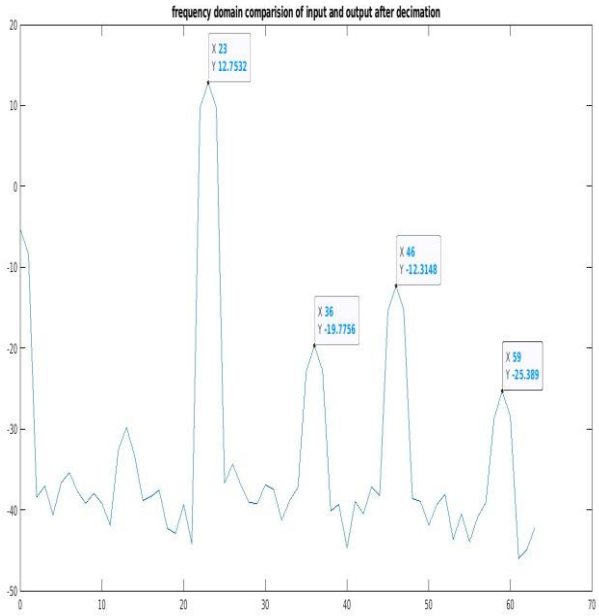

Fig 4:PSD of decimated signal (with mismatch)

Applying DWA technique to mitigate effect of mismatch in elements

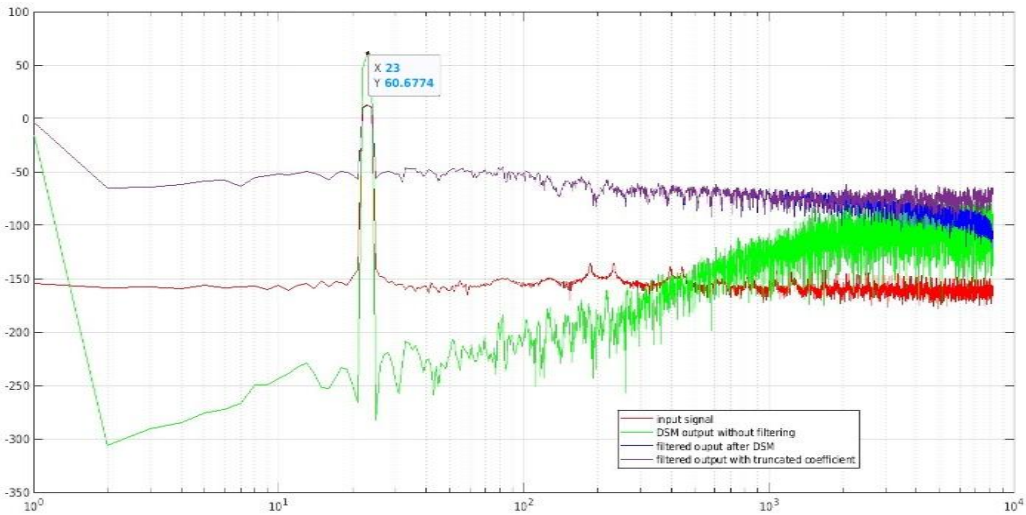

Fig 5:PSD of output signals after applying DWA

DWA is a dynamic element matching technique used in multi-bit ΔΣ modulators to reduce mismatch errors in DAC elements.
It cyclically rotates the usage of unit elements so that mismatch error is shaped like high-frequency noise instead of appearing as distortion.
This improves linearity and pushes mismatch noise out of band, enhancing overall SNR and SFDR.

```
snr_manual_calculated =

  107.9315

snr_after_filtering =

  107.0343

snr_after_filtering_trunc =

  107.0221

snr_after_decimation =

  109.1596

snr_after_simulation =

  134.4206
```

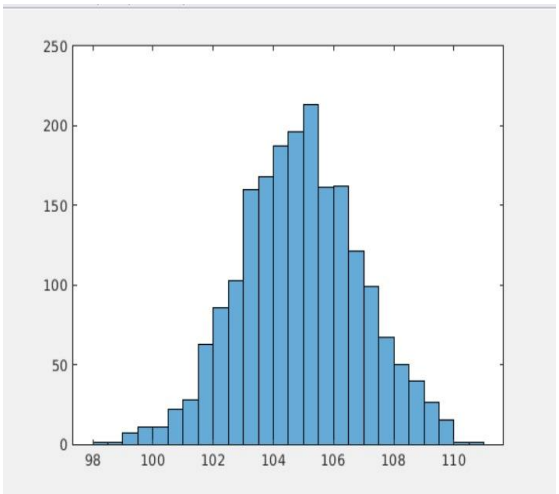

Fig 6:SNR distribution

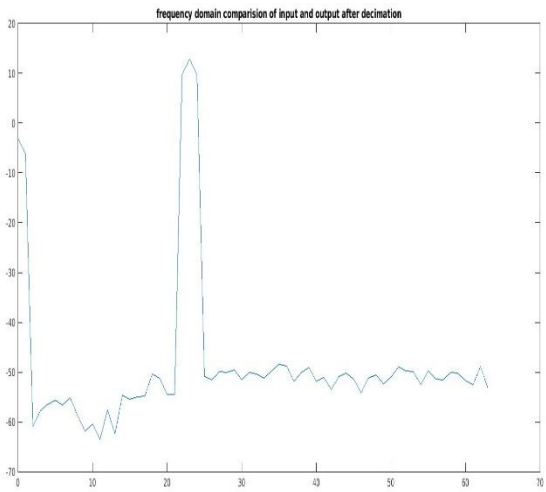

Fig 7 :PSD of Decimated signal (with DWA)

CIC filter implementation

```
%% Stage 1 : CIC (R = 32, N = 3)
cic1 = dsp.CICDecimator(32,1,3);
cic1_out = cic1(y.');

%% Stage 2 : CIC (R = 2, N = 3)
cic2 = dsp.CICDecimator(2,1,3);
cic2_out = cic2(cic1_out);

%% Stage 3 : Halfband FIR (R = 2)
hb = designfilt('lowpassfir',...
    'FilterOrder',44,...
    'HalfPowerFrequency',0.25,...
    'DesignMethod','maxflat');

hb_out = filter(hb,cic2_out);
```

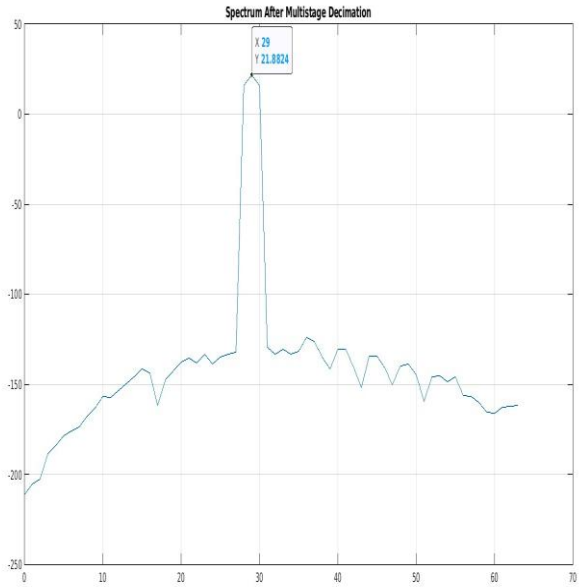

Fig 7 :PSD of Decimated signal(CIC filter)